

```
In [ ]: # 4.1 Случайный лес (классификация).
```

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
In [ ]: ## 1. Загрузка и обработка датасета.
Датасет, его загрузка и обработка как в заданиях 1.1, 2,1, 3.1.
```

```
In [2]: df = pd.read_csv('C:/Users/Elizaveta/Downloads/magic_gamma/telescope_data.csv')
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
df.head(5)
df.info()
print("размер", df.shape)
print("названия столбцов", list(df.columns))
print("уникальные значения в class", df['class'].unique())
print("\nраспределение классов")
print(df['class'].value_counts())
print("доли классов")
print(df['class'].value_counts(normalize=True))
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19020 entries, 0 to 19019
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Unnamed: 0    19020 non-null  int64
1   fLength      19020 non-null  float64
2   fWidth       19020 non-null  float64
3   fSize        19020 non-null  float64
4   fConc        19020 non-null  float64
5   fConc1       19020 non-null  float64
6   fAsym        19020 non-null  float64
7   fM3Long      19020 non-null  float64
8   fM3Trans     19020 non-null  float64
9   fAlpha       19020 non-null  float64
10  fDist        19020 non-null  float64
11  class        19020 non-null  object
dtypes: float64(10), int64(1), object(1)
memory usage: 1.7+ MB
размер (19020, 12)
названия столбцов ['Unnamed: 0', 'fLength', 'fWidth', 'fSize', 'fConc', 'fConc1',
'fAsym', 'fM3Long', 'fM3Trans', 'fAlpha', 'fDist', 'class']
уникальные значения в class ['g' 'h']
```

```
распределение классов
class
g    12332
h     6688
Name: count, dtype: int64
доли классов
class
g    0.64837
h    0.35163
Name: proportion, dtype: float64
```

```
In [3]: df['class']=df['class'].replace({'g': 1, 'h': 0})
print("first 10 values", df['class'].head(10))
print("unique values", df['class'].unique())
print("value counts", df['class'].value_counts())
print("type of data in class", df['class'].dtype)
print("first value", repr(df['class'].iloc[0]))
print("NaN", df['class'].isna().sum())
df=df.drop('Unnamed: 0', axis=1)
```

```

first 10 values 0    1
1    1
2    1
3    1
4    1
5    1
6    1
7    1
8    1
9    1
Name: class, dtype: int64
unique values [1 0]
value counts class
1    12332
0     6688
Name: count, dtype: int64
type of data in class int64
first value np.int64(1)
NaN 0

```

C:\Users\Elizaveta\AppData\Local\Temp\ipykernel_21812\2620301820.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer\_objects(copy=False)`. To opt-in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

```
df['class']=df['class'].replace({'g': 1, 'h': 0})
```

```

In [4]: X=df.drop('class', axis=1)
        Y=df['class']
        print("size of X", {X.shape})
        print("size of Y", {Y.shape})
        X_train, X_test, Y_train, Y_test = train_test_split(
            X, Y,
            test_size=0.3,
            random_state = 42,
            stratify = Y)
        print(f"size of X_train", {X_train.shape})
        print(f"size of X_test", {X_test.shape})
        print(f"size of Y_train", {Y_train.shape})
        print(f"size of Y_test", {Y_test.shape})
        print("balance of classes:")
        print(f"start data: 1={Y.mean():.3f}, 0={1-Y.mean():.3f}")
        print(f"train data: 1={Y_train.mean():.3f}, 0={1-Y_train.mean():.3f}")
        print(f"start data: 1={Y_test.mean():.3f}, 0={1-Y_test.mean():.3f}")

```

```

size of X {(19020, 10)}
size of Y {(19020,)}
size of X_train {(13314, 10)}
size of X_test {(5706, 10)}
size of Y_train {(13314,)}
size of Y_test {(5706,)}
balance of classes:
start data: 1=0.648, 0=0.352
train data: 1=0.648, 0=0.352
start data: 1=0.648, 0=0.352

```

```

In [ ]: # 2. Создание бейзлайна.
        Обучение модели RandomForestClassifier из библиотеки sklearn.
        F1 учитывает оба класса и их ошибки, показывает точность (сколько из отнесённых
        и полноту (сколько найдено настоящих объектов класса) классификации. В случае, е
        при высоком Accurasy, F1 покажет это.

```

Accuracy показывает, какую долю объектов модель классифицировала правильно. В да поэтому эта метрика даст адекватную оценку.
 Обучающая выборка (train) используется для обучения модели.
 Тестовая выборка (test) используется для оценки качества обученной модели.
 Результат на бейзлайне: F1=0.9124, accuracy=0.8828.
 Получены достаточно хорошие результаты, но есть потенциал для улучшения. Эти дан отталкиваться в сторону улучшений, показывающей, чего можно достичь минимальными

```
In [17]: from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score, accuracy_score
RForest_classifier = RandomForestClassifier(n_estimators=100)
RForest_classifier.fit(X_train, Y_train)
Y_prediction = RForest_classifier.predict(X_test)
F1 = f1_score(Y_test, Y_prediction)
accuracy = accuracy_score(Y_test, Y_prediction)
print(f" F1={F1:.4f}, accuracy={accuracy:.4f}")
```

F1=0.9124, accuracy=0.8828

```
In [ ]: ##3. Улучшение бейзлайна.
Гипотеза 1: Подбор гиперпараметров.
Оптимизируем гиперпараметры с помощью RandomSearchCV: число деревьев, максимальн
признаков для каждого разбиения, критерий разделения, балансировка классов. Случ
тестовых данных. Результаты:
Лучшие параметры {'n_estimators': 500, 'min_samples_split': 2, 'min_samples_leaf
Лучший F1 на CV: 0.9082039787161301
Тестовые метрики: F1 = 0.9148, Accuracy = 0.8857
Улучшения метрик практически отсутствуют, это говорит о том, что случайный лес с
данных.
```

```
In [18]: from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
import warnings
warnings.filterwarnings('ignore')
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
param_dist_clf = {
    'n_estimators': [100, 200, 300, 500],
    'max_depth': [10, 20, 30, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', 0.3, 0.5],
    'criterion': ['gini', 'entropy'],
    'class_weight': [None, 'balanced']
}
rf_clf = RandomForestClassifier(random_state=42, n_jobs=-1)
random_search_clf = RandomizedSearchCV(
    rf_clf,
    param_distributions=param_dist_clf,
    n_iter=50,
    cv=cv,
    scoring='f1',
    random_state=42,
    n_jobs=-1,
    verbose=1
)
random_search_clf.fit(X_train, Y_train)
print("Лучшие параметры", random_search_clf.best_params_)
print("Лучший F1 на CV:", random_search_clf.best_score_)
best_rf_clf = random_search_clf.best_estimator_
best_rf_clf.fit(X_train, Y_train)
y_pred = best_rf_clf.predict(X_test)
```

```
f1_test = f1_score(Y_test, y_pred)
accuracy_test = accuracy_score(Y_test, y_pred)
print(f"Тестовые метрики: F1 = {f1_test:.4f}, Accuracy = {accuracy_test:.4f}")
```

Fitting 5 folds for each of 50 candidates, totalling 250 fits

Лучшие параметры {'n_estimators': 500, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'sqrt', 'max_depth': None, 'criterion': 'gini', 'class_weight': None}

Лучший F1 на CV: 0.9082039787161301

Тестовые метрики: F1 = 0.9148, Accuracy = 0.8857

In []: *## 4. Имплементация алгоритма машинного обучения.*

Был создан собственный алгоритм случайного леса на основе ранее разработанного ансамблевого метода, строящий множество решающих деревьев на основе различных признаков для каждого разбиения, что снижает корреляцию между деревьями. Основные:

- `__init__` - задание гиперпараметров.
- `fit` - строит лес из `n_estimators` деревьев.
- `_impurity` - вычисляет меру неопределённости для массива меток.
- `predict` - формирование предсказаний.

```
In [19]: class Node:
    def __init__(self, feature=None, threshold=None, left=None, right=None, proba=None):
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.proba = proba

class MyDecisionTreeClassifier:
    def __init__(self, max_depth=None, min_samples_split=2, min_samples_leaf=1,
                  criterion='gini', max_features=None, random_state=None):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf
        self.criterion = criterion.lower()
        self.max_features = max_features
        self.random_state = random_state
        self.root = None
        self.n_classes_ = None
        self.n_features_ = None
        self.rng = np.random.RandomState(random_state)

    def fit(self, X, y):
        X = np.array(X)
        y = np.array(y).astype(int)
        self.n_classes_ = len(np.unique(y))
        self.n_features_ = X.shape[1]
        self.root = self._build_tree(X, y, depth=0)
        return self

    def _build_tree(self, X, y, depth):
        n_samples = X.shape[0]
        n_classes = len(np.unique(y))
        if (depth == self.max_depth) or (n_samples < self.min_samples_split) or \
            (n_classes < self.min_samples_leaf):
            proba = np.bincount(y, minlength=self.n_classes_) / len(y)
            return Node(proba=proba)
        best_feat, best_thresh, best_gain = self._best_split(X, y)
        if best_feat is None:
            proba = np.bincount(y, minlength=self.n_classes_) / len(y)
            return Node(proba=proba)
        left_mask = X[:, best_feat] <= best_thresh
        right_mask = ~left_mask
        left_child = self._build_tree(X[left_mask], y[left_mask], depth + 1)
        right_child = self._build_tree(X[right_mask], y[right_mask], depth + 1)
        return Node(feature=best_feat, threshold=best_thresh, left=left_child, right=right_child, proba=None)
```

```

        right_child = self._build_tree(X[right_mask], y[right_mask], depth + 1)
        return Node(feature=best_feat, threshold=best_thresh, left=left_child, right=right_child)

    def _best_split(self, X, y):
        best_gain = -float('inf')
        best_feature = None
        best_threshold = None
        current_impurity = self._impurity(y)
        n_features = X.shape[1]
        if self.max_features is None or self.max_features >= n_features:
            features_to_try = range(n_features)
        else:
            features_to_try = self.rng.choice(n_features, size=self.max_features, replace=True)
        for feature in features_to_try:
            thresholds = np.unique(X[:, feature])
            for thresh in thresholds:
                left_mask = X[:, feature] <= thresh
                right_mask = ~left_mask
                if np.sum(left_mask) < self.min_samples_leaf or np.sum(right_mask) < self.min_samples_leaf:
                    continue
                left_impurity = self._impurity(y[left_mask])
                right_impurity = self._impurity(y[right_mask])
                n_left = np.sum(left_mask)
                n_right = np.sum(right_mask)
                n_total = n_left + n_right
                gain = current_impurity - (n_left / n_total) * left_impurity - (n_right / n_total) * right_impurity
                if gain > best_gain:
                    best_gain = gain
                    best_feature = feature
                    best_threshold = thresh
        if best_gain <= 0:
            return None, None, None
        return best_feature, best_threshold, best_gain

    def _impurity(self, y):
        if len(y) == 0:
            return 0
        probs = np.bincount(y, minlength=self.n_classes_) / len(y)
        if self.criterion == 'gini':
            return 1 - np.sum(probs**2)
        else:
            probs = probs[probs > 0]
            return -np.sum(probs * np.log2(probs))

    def predict_proba(self, X):
        X = np.array(X)
        probas = []
        for x in X:
            node = self.root
            while node.proba is None:
                if x[node.feature] <= node.threshold:
                    node = node.left
                else:
                    node = node.right
            probas.append(node.proba)
        return np.array(probas)

    def predict(self, X):
        proba = self.predict_proba(X)
        return np.argmax(proba, axis=1)

```

```

In [20]: class MyRandomForestClassifier:
        def __init__(self, n_estimators=100, max_depth=None, min_samples_split=2,
                    min_samples_leaf=1, criterion='gini', max_features='sqrt',

```

```

        bootstrap=True, random_state=None):
self.n_estimators = n_estimators
self.max_depth = max_depth
self.min_samples_split = min_samples_split
self.min_samples_leaf = min_samples_leaf
self.criterion = criterion
self.max_features = max_features
self.bootstrap = bootstrap
self.random_state = random_state
self.trees = []
def fit(self, X, y):
X = np.array(X)
y = np.array(y)
n_samples, n_features = X.shape
self.classes_ = np.unique(y)
if isinstance(self.max_features, str):
    if self.max_features == 'sqrt':
        max_features_val = int(np.sqrt(n_features))
    elif self.max_features == 'log2':
        max_features_val = int(np.log2(n_features))
    else:
        raise ValueError("max_features должен быть 'sqrt', 'log2' или чи
elif isinstance(self.max_features, float):
    max_features_val = int(self.max_features * n_features)
else:
    max_features_val = self.max_features
if max_features_val is None or max_features_val > n_features:
    max_features_val = n_features
if max_features_val < 1:
    max_features_val = 1
rng = np.random.RandomState(self.random_state)
for i in range(self.n_estimators):
    if self.bootstrap:
        indices = rng.choice(n_samples, size=n_samples, replace=True)
    else:
        indices = np.arange(n_samples)
    X_boot = X[indices]
    y_boot = y[indices]
    tree = MyDecisionTreeClassifier(
        max_depth=self.max_depth,
        min_samples_split=self.min_samples_split,
        min_samples_leaf=self.min_samples_leaf,
        criterion=self.criterion,
        max_features=max_features_val,
        random_state=rng.randint(0, 10**6)
    )
    tree.fit(X_boot, y_boot)
    self.trees.append(tree)
return self
def predict_proba(self, X):
X = np.array(X)
probas = []
for tree in self.trees:
    proba = tree.predict_proba(X)
    probas.append(proba)
return np.mean(probas, axis=0)
def predict(self, X):
proba = self.predict_proba(X)
return np.argmax(proba, axis=1)

```

In []: Тестирование собственного алгоритма с гиперпараметрами, найденными для базовой модели. Полученные результаты практически идентичны результатам работы библиотечной модели.
F1 = 0.9145, accuracy = 0.8858.

```
In [ ]: my_rf_clf = MyRandomForestClassifier(  
    n_estimators=500,  
    max_depth=None,  
    min_samples_split=2,  
    min_samples_leaf=1,  
    criterion='gini',  
    max_features='sqrt',  
    bootstrap=True,  
    random_state=42  
)  
my_rf_clf.fit(X_train, Y_train)  
y_pred = my_rf_clf.predict(X_test)  
print("MyRandomForest:")  
print(f"F1 = {f1_score(Y_test, y_pred):.4f}, accuracy = {accuracy_score(Y_test,
```

In []: ## Выводы:
В ходе исследования алгоритма случайного леса для классификации была обучена базовая модель. Для улучшения базовой модели применялся подбор гиперпараметров. Результаты тестирования показали, что подобранные гиперпараметры не улучшили метрики. F1 = 0.9124, accuracy = 0.8828. Для улучшения базовой модели применялся подбор гиперпараметров. Результаты тестирования показали, что подобранные гиперпараметры не улучшили метрики. F1 = 0.9148, Accuracy = 0.8857. Улучшения метрик практически отсутствуют, это говорит о том, что базовая модель работает хорошо. Далее был создан собственный алгоритм случайного леса для регрессии на основе данных. Для улучшения собственного алгоритма были получены значения метрик F1 = 0.9145, accuracy = 0.8858, что говорит о корректной реализации алгоритма.